INFORME TÉCNICO DE ARQUITECTURA

SIMULADOR DE PLANIFICACIÓN DE PROCESOS MULTIHILO POSIX

Curso de Sistemas Operativos | C++17 POSIX Threads Engine

Asignatura / Proyecto	Sistemas Operativos — Plantilla Oficial de Entregable Técnico
Miembro 1 (Core / Concurrencia)	Santhiago Singer
Miembro 2 (CLI Shell)	Joan Briceño
Miembro 3 (CPU Scheduler)	Camilo Ceci

1. Arquitectura General del Sistema y Concurrencia (Módulo 1)

1.1 Modelo de Ejecución y Filosofía del Kernel

El simulador de planificación de procesos está desarrollado como un kernel multihilo concurrente alineado con el estándar **POSIX (IEEE Std 1003.1)** bajo **C++17**. La arquitectura establece una separación estricta entre la capa de interacción con el usuario (interfaz de línea de comandos o Shell) y el motor de procesamiento emulado (CPU y planificador de tareas).
El sistema opera mediante dos hilos independientes de ejecución creados al arranque de la aplicación:

- **Hilo CLI (cliThreadRoutine):** Atiende la lectura no bloqueante de comandos del usuario, la validación de sintaxis y el control de ejecución.
- **Hilo CPU (cpuThreadRoutine):** Emula el avance del reloj discreto del sistema (clock_ticks) y el consumo de ráfagas de CPU de los procesos.

1.2 Diagrama de Hilos y Sincronización POSIX

La tabla global de procesos (process_table) y las variables de estado del reloj (clock_ticks) residen en memoria compartida accesible por ambos hilos. Para evitar condiciones de carrera (race conditions), todo acceso de lectura o escritura a la tabla está resguardado mediante un cerrojo de exclusión mutua pthread_mutex_t (system_mutex), implementando el protocolo estricto de bloqueo pthread_mutex_lock y pthread_mutex_unlock.

1.3 Estructura Modular de Archivos

Archivo	Responsabilidad Principal	Componentes Clave
pcb.h	Definición del Process Control Block y estados del ciclo de vida.	struct PCB, enum class ProcessState, printSummary(), printDetail()
pcb.cpp	Lógica del constructor de PCB y formateo impreso de tablas.	Constructor PCB, stateToString(), métodos de consola
globals.h	Declaración de variables y estructuras compartidas (extern).	extern process_table, extern system_mutex, extern clock_ticks
globals.cpp	Instanciación real de variables compartidas y cerrojo mutex.	PTHREAD_MUTEX_INITIALIZER, findProcessByPID(), cleanupProcesses()
cli.h	Declaración de la interfaz en consola y el punto de entrada del hilo del CLI	cliThreadRoutine()
cli.cpp	Motor de parsing, validación estricta de sintaxis, tipos y	std::stringstream (validación de comandos como create,

Archivo	Responsabilidad Principal	Componentes Clave
	ejecución de comandos de la consola	load, ps, etc..). Llamadas thread-safe con pthread_mutex (comando exit)
cpuscheduler.h	Declaración del motor de planificación de la cpu, estructuras del historial de ejecución para realizar diagramas de gantt y prototipos de funciones globales	enum class SchedAlgorithm struct GanttSegment setSchedulerPolicy() cpuThreadRoutine() printGranttDiagram() printSystemStats()
cpuscheduler.cpp	Lógica del hilo motor: simulación del reloj discreto, algoritmos RR y SRTF, desalojo, cálculo de métricas (WT, TAT, RT) y generación del diagrama de Gantt	cpuThreadRoutine() std::this_thread::sleep_for() printGranttDiagram()
main.cpp	Orquestación multihilo POSIX y ciclo de vida de la aplicación.	pthread_create(), pthread_join(),

1.4 Guía de Inicio Rápido y Entorno POSIX (Build Guide)

El proyecto requiere un entorno compatible con C++17 y la biblioteca de hilos POSIX (pthread.h). En sistemas Windows se utiliza la distribución GCC 14.x (WinLibs / MinGW-w64 POSIX), mientras que en Linux/WSL y macOS se emplea la cadena de herramientas nativa g++ / clang++.

- **Comandos de Compilación Multihilo:**
    - **Compilación:** `make`
    - **Limpieza:** `make clean`

- **Comando de Ejecución:**
    - `./simulador.exe`

1.5 Metodología de Trabajo y Flujo en Git (GitHub & Kanban)

Para garantizar la calidad del software y cumplir con las prácticas profesionales de desarrollo en equipo, el desarrollo se organiza mediante un flujo estricto de GitHub Flow y Kanban:

- **Estrategia de Ramas (Branching):** Queda prohibido realizar commits directos sobre la rama main. Cada integrante desarrolla su módulo en una rama de característica independiente (ej. feature/concurrencia-pcb, feature/cli-parser, feature/scheduler-rr).
- **Integración mediante Pull Requests (PR):** Toda fusión a main requiere la creación de un PR en GitHub, revisión de código y la aprobación explícita (Approve) de al menos otro integrante del equipo.
- **Trazabilidad Kanban y Cierre de Issues:** Las tareas se gestionan en el tablero del repositorio mediante columnas (To Do, In Progress, Done). Los mensajes de commit incluyen palabras clave de vinculación (ej. Closes #1, Closes #2) para transicionar automáticamente los issues al completarse.

2. Interfaz de Consola e Intérprete de Comandos CLI (Módulo 2)

2.1 Arquitectura del Parser de Comandos implementado en el Hilo CLI

El intérprete de comandos (CLI) se ejecuta como un hilo POSIX independiente (**cliThreadRoutine()**). Para garantizar la robustez del sistema y evitar el colapso de la consola ante entradas anómalas (como múltiples espacios, caracteres especiales o pulsaciones de Enter vacías), la captura de datos no utiliza un std::cin directo. En su lugar la arquitectura emplea:

- **Captura de línea completa:** se utiliza std::getline(std::cin, line) para extraer de forma segura toda la cadena introducida por el usuario.
- **Uso dinámico de tokens:** la cadena capturada se procesa mediante un std::stringstream, el cual divide la línea en palabras individuales ignorando los espacios

adicionales.

- **Almacenamiento y enrutamiento:** los tokens resultantes se almacenan en un vector (std::vector<std::string> args). El primer elemento (args[0]) se evalúa en una estructura condicional para determinar la acción que debe ejecutarse, mientras que el resto del vector se procesa como los argumentos del comando.

2.2 Sintaxis y Validación

Cada comando implementa una validación de dos capas: cantidad de parámetros (tamaño del vector args) y seguridad de tipos (Type Safety mediante bloques try-catch y std::stoi).

- **create <nombre> <ráfaga> [prioridad]:** Instancia un nuevo proceso (PCB). La prioridad es opcional (por defecto es de 3). Exige 3 o 4 parámetros, valida mediante try-catch que la rafaga sea un entero n > 0 y que la prioridad esté en un rango entre 1 y 5.
- **load <archivo>**: Abre y lee un archivo de texto secuencial para ejecutar la carga por lotes de múltiples procesos del sistema. Cada línea del documento debe seguir la estructura base (<nombre> <ráfaga> [prioridad]).
    - **Validación:**
        - **Argumentos y Apertura:** Exige exactamente 2 parámetros en consola. Utiliza el método is_open() de la clase std::ifstream para verificar la existencia y accesibilidad del archivo en el disco; de fallar, aborta la operación de forma segura informando el error.
        - **Filtro de lectura:** El parser de archivo ignora automáticamente las líneas vacías y omite aquellas que inician con el carácter numeral (#), reservandolo para comentarios dentro de la traza, además reemplaza las comas (,) por espacios en blanco para que se procese correctamente.
          Ejemplo de formato esperado en el archivo:
          # NOMBRE, BURST_TIME, PRIORIDAD, ARRIVAL_TIME
          P1, 3, 3, 0
          P2, 1, 2, 1
          P1, 4, 1, 3
        - **Validación de Tipos por Línea:** Por cada línea procesada, exige exactamente 4 parámetros (Nombre, Burst_Time, Prioridad y Arrival_Time). Mediante bloques try-catch y std::stoi, valida que la ráfaga sea > 0, la prioridad esté entre 1 y 5, y el tiempo de llegada (Arrival_Time) sea >= 0.
        - **Tolerancia a Fallos:** Si una línea específica presenta un error de sintaxis o de formato numérico, el sistema emite una alerta indicando el número de línea afectada, omite su creación y continúa iterando el resto del archivo ininterrumpidamente.
        - **Exclusión Mutua:** La instanciación dinámica del objeto PCB y su inyección en la tabla de procesos global se realiza atómicamente inyectando el Arrival_Time extraído del archivo, encapsulando la escritura bajo el cerrojo system_mutex para evitar condiciones de carrera con el Hilo de CPU.
- **ps:** Imprime la tabla global de procesos. Rechaza la ejecución si se detectan

argumentos adicionales. Bloquea el system_mutex antes de iterar la memoria, no admite parámetros extra.

- **pcb <PID>:** Imprime la información detallada de un proceso específico. Exige exactamente 2 parámetros. El PID se valida con std::stoi asegurando que sea un número entero positivo.
- **set_sched <algoritmo> [quantum]:** configura y cambia en tiempo de ejecución la política de planificación que toman los procesos sobre la atención de la CPU entre RR y SRTF, donde se pide también el quantum que se va a dar y debe ser un número positivo entero.
- **set_speed <ms_por_tick>:** Ajusta el tiempo de retardo por ciclo de CPU. Exige 2 parámetros. Valida que el argumento sea un entero positivo y se actualiza simulation_speed_ms usando exclusión mutua.
- **start/pause:** Modifican el estado de ejecución del motor de la CPU. No admiten parámetros extra. Actualizan la variable global compartida (is_simulation_running) bajo protección de Mutex.
- **gantt:** imprime en consola la representación acumulada del diagrama de Gantt actual en texto, se protege la lectura de los bloques históricos mediante un mutex, no admite parámetros extra.
- **stats:** muestra las métricas globales de rendimiento del sistema, procesos finalizados, WT, TAT, RT y porcentaje de uso de la CPU, no admite parámetros extra.
- **help:** despliega el menú de ayuda de la CLI donde se explica la lista de comandos, no admite parámetros extra.
- **exit:** Cierra el simulador de forma segura. Rechaza argumentos adicionales.

2.3 Ejemplos de Salida en Consola

A continuación, se presentan los escenarios de ejecución exitosa y validaciones de error para cada comando:

- **create**
  Éxito:

  > create ProcesoA 15 2
  Proceso ‘ProcesoA’ creado con éxito (PID: 1).

  Errores de Sintaxis y Tipo:

  > create ProcesoA
  [ERROR DE SINTAXIS]: Parámetros insuficientes o excesivos.
  Uso correcto: create <nombre> <burst_time> [prioridad]

  > create ProcesoA -5
  [ERROR DE TIPO]: 'burst' debe ser entero positivo y 'prioridad' un entero entre 1 y 5.

- **load**
  Éxito:

  > load procesos.txt
  [ADVERTENCIA]: Línea 3 ignorada (formato incorrecto). Se esperaba:
  <nombre> <burst> [prioridad]

[ADVERTENCIA]: Línea 4 ignorada (tipos numéricos inválidos).
Carga por lotes exitosa. Se insertaron 5 procesos desde 'procesos.txt'.

Errores de Sintaxis y Archivo:

> load
[ERROR DE SINTAXIS]: Uso correcto: load <archivo>

> load no_existe.txt
[ERROR]: No se pudo abrir el archivo 'no_existe.txt'. Verifique la ruta y el nombre.

- **ps**

Éxito (Tabla de Procesos):

```
> ps
NOMBRE       ESTADO     PID    BURST  RESTANTE    PRIORIDAD
Chrome        READY      1              15s                2
Spotify      RUNNING     2               4s                4
```

Errores de Sintaxis:

> ps all
[ERROR DE SINTAXIS]: El comando 'ps' no recibe parámetros.

- **pcb <PID>**

Éxito (Detalle extendido):

```
> pcb 1
========================================
    CONTROL BLOCK DE PROCESO (PCB)
========================================
Nombre del Proceso : Chrome
PID                : 1
Estado Actual      : READY
Prioridad          : 2
Tiempo de Llegada  : 0 ticks
Ráfaga Inicial     : 15 ticks
Ráfaga Restante    : 15 ticks
Tiempo de Espera   : 3 ticks
Tiempo de Retorno  : N/A (En ejecución/pendiente)
Tiempo de Respuesta: N/A (No iniciado)
========================================
```

Errores de Sintaxis y Búsqueda:

> pcb uno
[ERROR DE TIPO]: El PID debe ser un número entero positivo.

> pcb 99
[ERROR]: No se encontró ningún proceso con el PID 99 en el sistema.

- **set_sched <algoritmo> [quantum]**

Éxito:

> set_sched RR 4
Algoritmo de planificación cambiado a RR (Quantum: 4).

Errores de Sintaxis y Parámetros:

> set_sched
[ERROR DE SINTAXIS]: Uso correcto: set_sched <RR|SRTF> [quantum]

> set_sched FCFS
[ERROR DE PARAMETRO]: Algoritmo no reconocido o quantum inválido. Algoritmos válidos: RR, SRTF.

- **set_speed <ms_por_tick>**
  Éxito:

  > set_speed 500
  Velocidad de simulación ajustada a 500 ms por tick.

  Errores de Sintaxis y Tipo:

  > set_speed rápido
  [ERROR DE TIPO]: La velocidad debe ser un número entero positivo en milisegundos.

- **start/pause**
  Éxito:

  > start
  Motor de Scheduling INICIADO/REANUDADO.

  > pause
  Motor de Scheduling PAUSADO.

  Errores de Sintaxis:

  > start now
  [ERROR DE SINTAXIS]: El comando 'start' no recibe parámetros.

- **gantt**
  Éxito (Con historial de procesos y tiempos ociosos):

  > gantt

  DIAGRAMA DE GANTT (0 - 10 ticks):
  [0-2s: IDLE] -> [2-5s: Chrome] -> [5-10s: Spotify]

  Errores de Sintaxis:

  > gantt ver
  [ERROR DE SINTAXIS]: El comando 'gantt' no recibe parámetros.

- **stats**
  Éxito:

  > stats

  ESTADÍSTICAS DEL SISTEMA:
  Procesos Finalizados : 2/5
  Tiempo Promedio de Espera : 4.50 ticks
  Tiempo Promedio de Retorno : 12.00 ticks
  Tiempo Promedio de Respuesta: 2.50 ticks
  Uso de CPU : 85.0%

Errores de Sintaxis:

> stats full
> [ERROR DE SINTAXIS]: El comando 'stats' no recibe parámetros.

- **help**
  Éxito:

> help
>
> --- MANUAL DE USUARIO (CLI) ---
> create <nombre> <burst> [prio] : Crea un nuevo proceso.
> load <archivo> : Lee un archivo de traza e inserta sus procesos
> ps : Despliega la tabla de procesos.
> pcb <PID> : Muestra detalle extendido del PCB.
> set_sched <algoritmo> [quantum] : Establece el algoritmo de planificacion (RR o SRTF)
> set_speed <ms_por_tick> : Ajusta la velocidad del reloj.
> start : Inicia/reanuda el Motor de CPU.
> pause : Pausa el reloj de la CPU.
> gantt : Imprime diagrama de Gantt actual.
> stats : Muestra métricas globales del sistema.
> help : Muestra este manual.
> exit : Libera memoria y cierra el simulador.
> ---

- **exit**
  Éxito al salir:

> > exit
> > Cerrando la simulación y liberando memoria compartida...

  Comando inexistente:

> > limpiar
> > [ERROR]: Comando no reconocido ('limpiar'). Escriba 'help' para ver el manual.

2.4 Apagado Ordenado

El comando **exit** está diseñado para garantizar que no existan fugas de memoria (memory leaks) ni procesos huérfanos al cerrar el programa.

Cuando el usuario ingresa este comando:

- Se activa una bandera global de control (por ejemplo, system_running = false) que notifica al Hilo Motor que debe detener su bucle de ejecución, permitiendo que finalice de forma natural.
- Se invoca la función cleanupProcesses(), la cual recorre la tabla global de procesos e invoca el operador delete sobre cada puntero PCB instanciado dinámicamente.
- El hilo CLI rompe su bucle principal y retorna la ejecución del orquestador (función main), logrando un cierre limpio y coordinado de ambos hilos del simulador.

3. Motor de Planificación de CPU y Algoritmos (Módulo 3)

Para simular el reloj discreto del sistema operativo (clock_ticks) no se mide usando el reloj interno de la máquina sino que para mantener el rigor de los cálculos se utilizó una variable entera acumulativa llamada clock_ticks que se va incrementando +1 cada vez que se cumple un ciclo de CPU consumido.

Con el objetivo de poder tener una visualización de cómo funcionan los sistemas operativos más didácticamente se tiene otra variable que controla la velocidad a la que se reproducen los ticks de reloj, por lo que asignamos la cantidad de ms por la que se “duerme” entre ticks mediante el set_speed, por defecto es 1000ms = 1s.

El hilo motor de la CPU simula la llegada diferida de procesos evaluando el tiempo de llegada (arrival_time) contra la variable discreta del reloj (clock_ticks). Los procesos cuyo tiempo de llegada sea mayor al tiempo actual permanecen retenidos hasta que el reloj alcance dicho tick, momento en el cual transicionan a la cola de listos (READY).
El cálculo del tiempo de espera de un proceso considera explícitamente el instante de llegada real:

WT = (T_completion - Arrival_Time) - Burst_initial

- Round Robin (RR): el round robin es una política de planificación apropiativa que busca la equidad de atención basada en una misma cantidad de atención para todos los procesos, la cual es el quantum. El planificador extrae secuencialmente los procesos de la cola de listos y les asigna la CPU por el intervalo del quantum, (current_quantum) que por defecto es 2 ticks, para gestionar el ciclo mientras que el proceso se ejecuta, un contador interno del quantum (quantum_counter) se incrementa por cada tick, si el quantum se agota y el proceso aún le resta ráfaga por consumir (remaining_burst > 0), este es desapropiado de la CPU, cambia su estado a READY y se le manda al final de la cola de listos para garantizar asi una distribución equitativa.

- Shortest Remaining Time First (SRTF): esta es la versión apropiativa del SFJ y su criterio de selección consta en que en cada ciclo del reloj (tick en nuestro caso), el planificador evalúa de manera global todos los procesos disponibles en la cola de listos junto con el proceso en ejecución y selecciona aquel que posea el menor

tiempo de ráfaga restante (remaining_burst). Si al sistema ingresa un nuevo proceso o un proceso en espera reduce su ráfaga y se detecta uno con tiempo menor al que se está ejecutando actualmente, el planificador aplica desapropiación inmediatamente. El proceso en ejecución regresa al estado READY y el nuevo proceso óptimo toma el control de la CPU instantáneamente.

El ciclo de vida de los procesos modelados en el PCB se rige bajo las siguientes transiciones supervisadas por el planificador y protegidas por la exclusión mutua (pthread_mutex):

- **NEW -> READY:** ocurre cuando el proceso se crea en el sistema y el reloj discreto alcanza o supera su tiempo de llegada de registro (arrival_time <= clock_ticks).
- **READY -> RUNNING:** el planificador selecciona el proceso de la cola de listos según la política que esté activa y le otorga la CPU. Si es su primer ejecución se calcula el tiempo de respuesta RT.
- **RUNNING-> TERMINATED:** se ejecuta cuando el consumo de la ráfaga llega a su fin (remaining_burst == 0). En este punto se registran el tiempo de finalización (completion_time) y el tiempo de retorno TAT.
- **RUNNING -> READY:** esto es el desalojo de la CPU y ocurre bajo dos escenarios, el primero es si se agota el quantum bajo la política del round robin y el segundo es la llegada de un proceso más corto bajo la política SRTF.

La siguiente tabla demuestra cómo el parámetro de velocidad afecta la experiencia de observación en tiempo real sin alterar las métricas matemáticas internas del simulador

Parámetro (set_speed)	Retardo real por tick (sleep)	Duración real de 5 ticks de la CPU (segundos reales de espera)	Comportamiento del sistema
50ms	50ms	0,25s	Simulación ultrarrápida, para pruebas automatizadas de lotes grandes de procesos
1000ms	1s	5s	Simulación estándar en cámara lenta controlada que permite la inspección mediante comandos

5000ms	5s	25s	Simulación muy lenta y pausada que permite demostrar paso a paso el funcionamiento y validar las interrupciones

4. Carga Batch de Procesos y Evaluación de Métricas (Módulo 4)

4.1 Definición Matemática de Métricas de Planificación

Para evaluar la eficiencia del kernel y comparar el comportamiento de los algoritmos de planificación (Round Robin y SRTF), el simulador registra el avance del tiempo discreto mediante ticks del reloj (clock_ticks) y calcula automáticamente las siguientes métricas fundamentales para cada PCB instanciado:

1. **Tiempo de Retorno / Permanencia (*Turnaround Time - TAT*):** Es el tiempo total transcurrido desde que un proceso ingresa al sistema (estado NEW / READY) hasta que completa totalmente su ráfaga de ejecución (estado TERMINATED).

$$TAT = T_{completion} - T_{arrival}$$

2. **Tiempo de Espera (*Waiting Time - WT*):** Es el tiempo total acumulado que un proceso permanece en la cola de listos (READY) esperando que se le asigne la CPU. Se calcula restando la ráfaga total ejecutada al tiempo de retorno.

$$WT = TAT - Burst_{initial}$$

3. **Tiempo de Respuesta (*Response Time - RT*):** Es el intervalo de tiempo desde que el proceso llega a la cola de listos hasta que obtiene el procesador por primera vez y ejecuta su primera instrucción.

$$RT = T_{first_run} - T_{arrival}$$

4. Porcentaje de uso del CPU: se calcula analizando todo el historial del vector gantt_log. Suma la duración de todos los segmentos donde la CPU estuvo ocupada por procesos y lo divide entre el total de clock_ticks transcurridos hasta el momento,

multiplicando por 100 para mostrar el porcentaje.

$$CPU\% = \frac{\sum Tburst}{clocktickactual - clocktickinicial}$$

4.2 Promedios Globales del Sistema

Al finalizar la ejecución de un conjunto de procesos o al invocar el comando stats en la CLI, el kernel calcula las métricas macro del sistema promediando los valores individuales de todos los procesos finalizados (N):

- **Tiempo Promedio de Espera:** $\bar{WT} = \frac{1}{N}\sum_{i=1}^{N} WT_i$
- **Tiempo Promedio de Retorno:** $\bar{TAT} = \frac{1}{N}\sum_{i=1}^{N} TAT_i$
- **Tiempo Promedio de Respuesta:** $\bar{RT} = \frac{1}{N}\sum_{i=1}^{N} RT_i$

4.3 Análisis Comparativo Teórico (RR vs. SRTF)

El comportamiento de las métricas varía de manera determinista según la política de planificación activa:

- **Round Robin (RR):** Garantiza un tiempo de respuesta ($\bar{RT}$) bajo y equitativo para todos los procesos debido al reparto periódico del procesador limitado por el Quantum. Sin embargo, en lotes con procesos de ráfagas heterogéneas, puede aumentar el tiempo promedio de espera ($\bar{WT}$) por el exceso de cambios de contexto.
- **Shortest Remaining Time First (SRTF):** Es matemáticamente óptimo para minimizar el tiempo promedio de espera ($\bar{WT}$) y el tiempo de retorno ($\bar{TAT}$) al dar prioridad a las ráfagas cortas. Su desventaja es el riesgo de inanición (*starvation*) para procesos con ráfagas extensas si existe una entrada continua de procesos cortos.

4.4 Resultados Experimentales y Análisis Comparativo Reales

A continuación se tabulan las métricas obtenidas tras procesar el lote de pruebas `procesos.txt` (4 procesos activos: `procesoA`, `procesoB`, `procesoC` y `procesoD`) bajo las dos políticas de planificación del simulador:

Algoritmo	Procesos Finalizados	Tiempo Promedio de Espera ($\overline{WT}$)	Tiempo Promedio de Retorno ($\overline{TAT}$)	Tiempo Promedio de Respuesta ($\overline{RT}$)	Uso de CPU (UCPU)
Round Robin (RR, $Q = 2$)	4 / 4	13.25 ticks	20.25 ticks	3.00 ticks	56.0%
SRTF (Apropiativo)	4 / 4	6.75 ticks	13.75 ticks	6.75 ticks	90.3%

4.5 Resultados Experimentales del archivo sugerido (con ARRIVAL_TIME)

A continuación se presentan los resultados obtenidos tras ejecutar el lote de prueba estandarizado workload_test.txt (Job_A, 4, 2, 0; Job_B, 2, 1, 1; Job_C, 1, 3, 3) bajo las políticas Round Robin y SRTF:

Algoritmo	Procesos Finalizados	WT Promedio	TAT Promedio	RT Promedio	Uso de CPU
Round Robin (RR, Q=2)	3 / 3	1.67 ticks	4.00 ticks	0.67 ticks	70.1%
SRTF (Apropiativo)	3 / 3	1.00 ticks	3.33 ticks	0 ticks	67.0%

5. Referencias Bibliográficas

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. IEEE Computer Society. (2018). *IEEE Std 1003.1-2017 (Revision of IEEE Std 1003.1-2008) - POSIX System Interfaces*.
3. ISO/IEC. (2017). *ISO/IEC 14882:2017 Programming languages — C++ (C++17 Standard)*.
4. Kerrisk, M. (2010). *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press.

Anexo A: Declaración del Uso Ético de Inteligencia Artificial

En cumplimiento con las directivas académicas de la asignatura, se declara el uso de asistentes de Inteligencia Artificial (Gemini AI) como herramienta colaborativa para la aceleración del desarrollo y diseño de la arquitectura. A continuación se detallan las dimensiones de uso:

- **Herramientas Empleadas:** Gemini AI Assistant.
- **Prompts Principales Ejecutados:** "Diseñar la estructura PCB en C++17 con enum de estados y funciones para imprimir comandos ps y pcb <PID>, junto con la inicialización de hilos pthread y mutex para evitar race conditions".

  "Actúa como un tutor socrático experto en C++17 y programación concurrente con pthreads. Estoy construyendo el parser interactivo (Shell-CLI) para un simulador de sistema operativo. Ayúdame a depurar mi bucle principal para evitar que la consola colapse con entradas anómalas utilizando std::getline y std::stringstream. Además, guíame en la implementación segura de exclusión mutua (pthread_mutex_t) cuando el hilo de la CLI deba leer o escribir en la tabla global de procesos, sin darme la solución completa de inmediato, sino explicando la lógica detrás de las secciones críticas."

  "Revisa la lógica de mi comando load. Necesito que actúe como un procesador por lotes leyendo un archivo .txt. Actúa como mi asistente de depuración: ¿cómo puedo utilizar <fstream> para limpiar las líneas, ignorar comas e inyectar exactamente 4 parámetros (Nombre, Ráfaga, Prioridad, Tiempo de Llegada) en el constructor del PCB, manteniendo el programa a prueba de fallos y documentando los errores de tipo en consola?"

“Actúa como tutor socrático para que me ayudes a plantear la estrategia y estructura que debo seguir al querer realizar un archivo en C++ para mi proyecto de sistemas operativos donde estamos simulando un entorno kernel, con sus rutinas para tener la vista de los procesos, el planificador del cpu para darle prioridad a un proceso u otro. Mi parte es la del planificador de la CPU, planteame que estructura de archivo debo seguir para que mis compañeros puedan acoplarse a ello sin ningún problema y sea funcional. Ten en cuenta que debemos calcular el tiempo de respuesta, tiempo de retorno y el tiempo de espera, además de calcular el porcentaje de uso de la CPU. Las políticas que vamos a aplicar para el planificador son un Round Robin con quantum variable y predeterminado de 1000 ms y SRTF. Con esta información cómo plantearías tú este entorno. Tal vez un archivo cpp para la funcionalidad, y uno .h para definir las estructuras que tenemos que compartir con mis compañeros de equipo.”

- **Proceso de Pensamiento Crítico y Validación:** El equipo evaluó rigurosamente cada componente sugerido por la IA, verificando la conformidad con el estándar POSIX (-pthread) y la especificación C++17. Se ajustaron manualmente las firmas de funciones de hilos para evitar advertencias de compilación (-Wreturn-type) y se comprobó experimentalmente la correcta exclusión mutua sobre la tabla global process_table en entornos WSL/Linux y Windows (GCC 14.x).

  Los bloques lógicos sugeridos por la herramienta fueron contrastados constantemente con la rúbrica del proyecto. Por ejemplo, se detectó que la IA inicialmente sugirió cargar 3 parámetros en el comando load, lo cual fue corregido e iterado por el equipo.
  Ningún código fue aceptado a ciegas. Cada fragmento de validación (como los bloques try-catch con std::stoi) se integró gradualmente y se compiló localmente utilizando el comando manual g++ *.cpp -o simulador.exe -std=c++17 -pthread (sugerido por la IA).
  Se validó de forma manual que los cerrojos pthread_mutex_lock y unlock propuestos rodearan estrictamente las secciones críticas, y se comprobó que el comando exit invocará el destructor de memoria dinámica (cleanupProcesses()) para garantizar el cierre seguro de los hilos.

NOTA: También le pedimos a gemini que nos hiciera la plantilla sobre la cual escribir, por eso los colores azules y la forma ordenada de navegar por el documento.